# Get, Clean Data

Gaurav Sood

# Data, Data, Everywhere

# Data, Data, Everywhere

– The Famous Five:
  Aural, Visual, Somatic, Gustatory, Olfactory

# Data, Data, Everywhere

- The Famous Five:
  Aural, Visual, Somatic, Gustatory, Olfactory
- The Social Famous Five:
  What people (like to) hear, see, sense, smell, taste, ...

# Data, Data, Everywhere

- The Famous Five:
  Aural, Visual, Somatic, Gustatory, Olfactory
- The Social Famous Five:
  What people (like to) hear, see, sense, smell, taste, ...
- Manifest Data:
  Likes, Ratings, Reviews, Comments, Views, Searches ...

# Data, Data, Everywhere

- The Famous Five:
  Aural, Visual, Somatic, Gustatory, Olfactory
- The Social Famous Five:
  What people (like to) hear, see, sense, smell, taste, ...
- Manifest Data:
  Likes, Ratings, Reviews, Comments, Views, Searches ...
- Data about data:
  Location of a tweet, photo, who called whom, ...

# Data, Data, Everywhere

- The Famous Five:
  Aural, Visual, Somatic, Gustatory, Olfactory
- The Social Famous Five:
  What people (like to) hear, see, sense, smell, taste, ...
- Manifest Data:
  Likes, Ratings, Reviews, Comments, Views, Searches ...
- Data about data:
  Location of a tweet, photo, who called whom, ...
- Social data:
  Friend graph, followers, who retweeted, liked,...

# Data, Data, Everywhere

- The Famous Five:
  Aural, Visual, Somatic, Gustatory, Olfactory
- The Social Famous Five:
  What people (like to) hear, see, sense, smell, taste, ...
- Manifest Data:
  Likes, Ratings, Reviews, Comments, Views, Searches ...
- Data about data:
  Location of a tweet, photo, who called whom, ...
- Social data:
  Friend graph, followers, who retweeted, liked,...
- Data about structure:
  Layout of the site, In/out links, ...

# Collecting Digital Data

– Proprietary Data collections
  Lexis-Nexis, comScore ...

# Collecting Digital Data

- Proprietary Data collections
  Lexis-Nexis, comScore ...
- APIs
  Facebook, NY Times, Twitter, Google, FourSquare, Jstor, Zillow ...

# Collecting Digital Data

- Proprietary Data collections
  Lexis-Nexis, comScore ...
- APIs
  Facebook, NY Times, Twitter, Google, FourSquare, Jstor, Zillow ...
- Bulk Downloads
  Wikipedia, data.gov, IMDB, Million Song Database, Google n-grams ...

# Collecting Digital Data

- Proprietary Data collections
  Lexis-Nexis, comScore ...
- APIs
  Facebook, NY Times, Twitter, Google, FourSquare, Jstor, Zillow ...
- Bulk Downloads
  Wikipedia, data.gov, IMDB, Million Song Database, Google n-grams ...
- Scraping

# Collecting Digital Data

- Proprietary Data collections
  Lexis-Nexis, comScore ...
- APIs
  Facebook, NY Times, Twitter, Google, FourSquare, Jstor, Zillow ...
- Bulk Downloads
  Wikipedia, data.gov, IMDB, Million Song Database, Google n-grams ...
- Scraping
- Custom Apps
  Build custom apps to observe behavior, get (pay) people to download these apps

# Scraping

– To analyze data, we typically need structure.
  For instance, same number of rows for each column.

# Scraping

- To analyze data, we typically need structure.
  For instance, same number of rows for each column.
- But found data often with human readable structure.
- Copy and paste, type, to a dataset.

# Scraping

- To analyze data, we typically need structure.
  For instance, same number of rows for each column.
- But found data often with human readable structure.
- Copy and paste, type, to a dataset.

# Scraping

- To analyze data, we typically need structure.
  For instance, same number of rows for each column.
- But found data often with human readable structure.
- Copy and paste, type, to a dataset.
- But error prone, and not scalable.

# Scraping

- To analyze data, we typically need structure.
  For instance, same number of rows for each column.
- But found data often with human readable structure.
- Copy and paste, type, to a dataset.
- But error prone, and not scalable.
- Idea: Find the less accessible structure, automate based on it.

# Collecting Found Digital Data

- Software

# Collecting Found Digital Data

- Software
  - R packages for HTTP requests and HTML parsing

# Collecting Found Digital Data

- Software
  - R packages for HTTP requests and HTML parsing
  - Python, Ruby, Perl, Java, ...

# Collecting Found Digital Data

- Software
  - R packages for HTTP requests and HTML parsing
  - Python, Ruby, Perl, Java, ...
  - Hosted scraping services

# Collecting Found Digital Data

- Software
  - R packages for HTTP requests and HTML parsing
  - Python, Ruby, Perl, Java, ...
  - Hosted scraping services
- Some things to keep in mind

# Collecting Found Digital Data

- Software
  - R packages for HTTP requests and HTML parsing
  - Python, Ruby, Perl, Java, ...
  - Hosted scraping services
- Some things to keep in mind
  - Check if there is an API, or if data are available for download

# Collecting Found Digital Data

- Software
  - R packages for HTTP requests and HTML parsing
  - Python, Ruby, Perl, Java, ...
  - Hosted scraping services
- Some things to keep in mind
  - Check if there is an API, or if data are available for download
  - Play Nice:

# Collecting Found Digital Data

- Software
  - R packages for HTTP requests and HTML parsing
  - Python, Ruby, Perl, Java, ...
  - Hosted scraping services
- Some things to keep in mind
  - Check if there is an API, or if data are available for download
  - Play Nice:
    - Scraper may be disallowed in 'robots.txt'

# Collecting Found Digital Data

- Software
  - R packages for HTTP requests and HTML parsing
  - Python, Ruby, Perl, Java, ...
  - Hosted scraping services
- Some things to keep in mind
  - Check if there is an API, or if data are available for download
  - Play Nice:
    - Scraper may be disallowed in 'robots.txt'
    - Respect documented rate limits and retry guidance.

# Collecting Found Digital Data

- Software
  - R packages for HTTP requests and HTML parsing
  - Python, Ruby, Perl, Java, ...
  - Hosted scraping services
- Some things to keep in mind
  - Check if there is an API, or if data are available for download
  - Play Nice:
    - Scraper may be disallowed in 'robots.txt'
    - Respect documented rate limits and retry guidance.
    - Scrape during off-peak hours

# Paper


TELEVISION & CABLE
FACTBOOK®
TV STATIONS 1996
Authoritative Reference for the Television, Cable & Electronics Industries
Published by Warren Publishing, Inc., 2115 Ward Court, N.W., Washington, D.C. 20037
Stations Volume No. 64 1996 Edition
Broadcasting's best friend is home again.
The RCA® broadcast equipment line is now part of Comark Communications Inc.
COMARK
THOMCAST

# Paper

- Create digital images of paper

# Paper

- Create digital images of paper
- Identify colored pixels as characters (OCR)

# Paper

- Create digital images of paper
- Identify colored pixels as characters (OCR)
- Software

# Paper

- Create digital images of paper
- Identify colored pixels as characters (OCR)
- Software
    - Adobe Pro., etc.

# Paper

- Create digital images of paper
- Identify colored pixels as characters (OCR)
- Software
  - Adobe Pro., etc.
  - Commercial OCR: ABBYY FineReader

# Paper

- Create digital images of paper
- Identify colored pixels as characters (OCR)
- Software
  - Adobe Pro., etc.
  - Commercial OCR: ABBYY FineReader
  - Open-source OCR: Tesseract

# Paper

- Create digital images of paper
- Identify colored pixels as characters (OCR)
- Software
  - Adobe Pro., etc.
  - Commercial OCR: ABBYY FineReader
  - Open-source OCR: Tesseract
- Extract the recognized text layer with pypdf

# Paper

- Create digital images of paper
- Identify colored pixels as characters (OCR)
- Software
  - Adobe Pro., etc.
  - Commercial OCR: ABBYY FineReader
  - Open-source OCR: Tesseract
- Extract the recognized text layer with pypdf
- Post-processing

# Pictures, Audio, and Video

- Audio (or Video with audio) to text: Dragon Dictates, Google transcription

# Pictures, Audio, and Video

- Audio (or Video with audio) to text: Dragon Dictates, Google transcription
- Pictures: recognize color, faces

# Pictures, Audio, and Video

- Audio (or Video with audio) to text: Dragon Dictates, Google transcription
- Pictures: recognize color, faces
- Objects in images: Clarifai

# Pictures, Audio, and Video

- Audio (or Video with audio) to text: Dragon Dictates, Google transcription
- Pictures: recognize color, faces
- Objects in images: Clarifai
- Scrape closed-captions

# Get Others to Work

- Human Computing

# Get Others to Work

- Human Computing
- Amazon.com's Mechanical Turk

# Get Others to Work

- Human Computing
- Amazon.com's Mechanical Turk
    - Create Human Intelligence Tasks (HITs)

# Get Others to Work

- Human Computing
- Amazon.com's Mechanical Turk
  - Create Human Intelligence Tasks (HITs)
  - Surveys, transcription, translation, ...

# Get Others to Work

- Human Computing
- Amazon.com's Mechanical Turk
  - Create Human Intelligence Tasks (HITs)
  - Surveys, transcription, translation, ...
  - You assess the work and pay out

# Get Others to Work

- Human Computing
- Amazon.com's Mechanical Turk
  - Create Human Intelligence Tasks (HITs)
  - Surveys, transcription, translation, ...
  - You assess the work and pay out
- Freelance platforms, impact sourcing, run your own ads ...

# Get Others to Work

- Human Computing
- Amazon.com's Mechanical Turk
  - Create Human Intelligence Tasks (HITs)
  - Surveys, transcription, translation, ...
  - You assess the work and pay out
- Freelance platforms, impact sourcing, run your own ads ...
- Provide clear tasks, fair payment, and quality checks.

# Scraping one HTML page in Python

Parse a small HTML document with Beautiful Soup 4.

```
from bs4 import BeautifulSoup

html = """<html><head><title>Twelfth Night</title></head>
<body><p>If music be the food of love, play on.</p>
<a href="https://example.com/">Example</a></body></html>"""
soup = BeautifulSoup(html, "html.parser")
text = soup.p.get_text()
print(text)
```

# Getting text from one pdf in Python

Run from the repository root. Install with `pip install -r requirements.txt`. Text extraction requires a text layer; scanned pages need OCR.

```
from pypdf import PdfReader

pdf_path = "ds3/ds3_present_web.pdf"
reader = PdfReader(pdf_path)
content = reader.pages[0].extract_text() or ""
print(content)
```

# Scraping many urls/files to structured data

- Loop, exploiting structure of the urls/file paths

# Scraping many urls/files to structured data

- Loop, exploiting structure of the urls/file paths
  e.g. ESPN URL

# Scraping many urls/files to structured data

- Loop, exploiting structure of the urls/file paths
  e.g. ESPN URL
- Handle errors, if files or urls don't open, what do you do?

# Scraping many urls/files to structured data

- Loop, exploiting structure of the urls/file paths
  e.g. ESPN URL
- Handle errors, if files or urls don't open, what do you do?
- To harvest structured data, exploit structure within text

# Scraping many urls/files to structured data

- Loop, exploiting structure of the urls/file paths
  e.g. ESPN URL
- Handle errors, if files or urls don't open, what do you do?
- To harvest structured data, exploit structure within text
- Trigger words, html tags, ...

# Exception(al) Handling

Catch errors you can handle; let unexpected failures surface.

```
from pypdf.errors import PdfReadError

try:
    reader = PdfReader(pdf_path)
except (OSError, PdfReadError) as error:
    print(f"Cannot open {pdf_path}: {error}")
```

# Inside the page

- Chrome Developer Tools

# Inside the page

- Chrome Developer Tools
- Quick Tour of HTML

# Inside the page

- Chrome Developer Tools
- Quick Tour of HTML
    - Tags begin with < and end with >

# Inside the page

- Chrome Developer Tools
- Quick Tour of HTML
  - Tags begin with < and end with >
  - Tags usually occur in pairs. Some don't (see img). And can be nested.

# Inside the page

- Chrome Developer Tools
- Quick Tour of HTML
  - Tags begin with < and end with >
  - Tags usually occur in pairs. Some don't (see img). And can be nested.
  - Mozilla HTML elements

# Inside the page

- Chrome Developer Tools
- Quick Tour of HTML
  - Tags begin with < and end with >
  - Tags usually occur in pairs. Some don't (see img). And can be nested.
  - Mozilla HTML elements
  - <p> is for paragraph

# Inside the page

- Chrome Developer Tools
- Quick Tour of HTML
  - Tags begin with < and end with >
  - Tags usually occur in pairs. Some don't (see img). And can be nested.
  - Mozilla HTML elements
  - <p> is for paragraph
  - <a> is for a link

# Inside the page

- Chrome Developer Tools
- Quick Tour of HTML
    - Tags begin with < and end with >
    - Tags usually occur in pairs. Some don't (see img). And can be nested.
    - Mozilla HTML elements
    - <p> is for paragraph
    - <a> is for a link
    - <ol>, <ul> is for ordered, unordered list; <li> is a bullet

# Inside the page

- Chrome Developer Tools
- Quick Tour of HTML
  - Tags begin with < and end with >
  - Tags usually occur in pairs. Some don't (see img). And can be nested.
  - Mozilla HTML elements
  - <p> is for paragraph
  - <a> is for a link
  - <ol>, <ul> is for ordered, unordered list; <li> is a bullet
  - tags can have attributes. <a href='http://somesite'></a>

# Inside the page

- Chrome Developer Tools
- Quick Tour of HTML
  - Tags begin with < and end with >
  - Tags usually occur in pairs. Some don't (see img). And can be nested.
  - Mozilla HTML elements
  - <p> is for paragraph
  - <a> is for a link
  - <ol>, <ul> is for ordered, unordered list; <li> is a bullet
  - tags can have attributes. <a href='http://somesite'></a>
  - DOM, hierarchical, parent, child:

```
<html>
    <body>
        <p></p>
    </body>
</html>
```

# Find Things

Use tags and attributes to find elements. Beautiful Soup documentation.

```
print(soup.title.get_text())
print(soup.find_all("a"))
print(soup.find(id="nav1"))
for link in soup.find_all("a", href=True):
    print(link["href"])
```

# Data Munging

"Data scientists, according to interviews and expert estimates, spend from 50 percent to 80 percent of their time mired in the mundane labor of collecting and preparing data, before it can be explored for useful information."

New York Times: For BigData Scientists, 'Janitor Work' Is Key Hurdle to Insights

# Data Munging

"In our experience, the tasks of exploratory data mining and data cleaning constitute 80% of the effort that determines 80% of the value of the ultimate data."

Dasu and Johnson, Exploratory Data Mining and Data Cleaning

# Regular (or Rational) Expressions

- Patterns for matching text; examples below use Python regex syntax.

# Regular (or Rational) Expressions

- Patterns for matching text; examples below use Python regex syntax.
- Stephen Kleene, 'inventor' of regular expressions.

# Regular (or Rational) Expressions

- Patterns for matching text; examples below use Python regex syntax.
- Stephen Kleene, 'inventor' of regular expressions.
- Henry Spencer, behind the `regex` library.

# Regular (or Rational) Expressions

- Patterns for matching text; examples below use Python regex syntax.
- Stephen Kleene, 'inventor' of regular expressions.
- Henry Spencer, behind the `regex` library.
- Descend from *finite automata* theory.

# Regular (or Rational) Expressions

- Patterns for matching text; examples below use Python regex syntax.
- Stephen Kleene, 'inventor' of regular expressions.
- Henry Spencer, behind the `regex` library.
- Descend from *finite automata* theory.
- Matching

# The most basic regular expression

- String literal

# The most basic regular expression

- String literal
- RegexPal.com
- Say you are searching for the word apple – can be uppercase first character, plural, lowercase first character

# Disjunction

- Disjunction, Character classes

# Disjunction

- Disjunction, Character classes
  - []

# Disjunction

- Disjunction, Character classes
  - `[]`
  - `[aA]pple matches apple and Apple`

# Disjunction

- Disjunction, Character classes
  - `[]`
  - `[aA]pple matches apple and Apple`
  - `[0123456789] matches an ASCII digit`

# Disjunction

- Disjunction, Character classes
  - `[]`
  - `[aA]pple matches apple and Apple`
  - `[0123456789] matches an ASCII digit`
- Ranges

# Disjunction

- Disjunction, Character classes
  - `[]`
  - `[aA]pple matches apple and Apple`
  - `[0123456789] matches an ASCII digit`
- Ranges
  - `[0-9] matches an ASCII digit`

# Disjunction

- Disjunction, Character classes
  - `[]`
  - `[aA]pple matches apple and Apple`
  - `[0123456789] matches an ASCII digit`
- Ranges
  - `[0-9] matches an ASCII digit`
  - `[a-z] matches an ASCII lowercase letter`

# Disjunction

- Disjunction, Character classes
  - `[]`
  - `[aA]pple matches apple and Apple`
  - `[0123456789] matches an ASCII digit`
- Ranges
  - `[0-9] matches an ASCII digit`
  - `[a-z] matches an ASCII lowercase letter`
  - `[a-zA-Z] matches an ASCII letter of either case`

# Disjunction

- Disjunction, Character classes
  - `[]`
  - `[aA]pple matches apple and Apple`
  - `[0123456789] matches an ASCII digit`
- Ranges
  - `[0-9] matches an ASCII digit`
  - `[a-z] matches an ASCII lowercase letter`
  - `[a-zA-Z] matches an ASCII letter of either case`
  - `[a-e1-9] matches a through e or 1 through 9`

# Disjunction

- Disjunction, Character classes
  - `[]`
  - `[aA]pple matches apple and Apple`
  - `[0123456789] matches an ASCII digit`
- Ranges
  - `[0-9] matches an ASCII digit`
  - `[a-z] matches an ASCII lowercase letter`
  - `[a-zA-Z] matches an ASCII letter of either case`
  - `[a-e1-9] matches a through e or 1 through 9`
  - Hyphen only has a special meaning if used within range.

    `[-123]`

# Disjunction Contd..

- Negation in Disjunction

# Disjunction Contd..

- Negation in Disjunction
  - `^ right after the square bracket means a negation`

# Disjunction Contd..

- Negation in Disjunction
  - `^ right after the square bracket means a negation`
  - `[^A-Z]`

# Disjunction Contd..

- Negation in Disjunction
  - `^ right after the square bracket means a negation`
  - `[^A-Z]`
  - `[^Aa] means neither a capital A nor a lowercase a`

# Disjunction Contd..

- Negation in Disjunction
  - `^ right after the square bracket means a negation`
  - `[^A-Z]`
  - `[^Aa] means neither a capital A nor a lowercase a`
  - `[^e^] means not an e, and not ^`

# Disjunction Contd..

- Negation in Disjunction
  - `^ right after the square bracket means a negation`
  - `[^A-Z]`
  - `[^Aa] means neither a capital A nor a lowercase a`
  - `[^e^] means not an e, and not ^`
- Disjunction for longer strings

# Disjunction Contd..

- Negation in Disjunction
  - `^ right after the square bracket means a negation`
  - `[^A-Z]`
  - `[^Aa] means neither a capital A nor a lowercase a`
  - `[^e^] means not an e, and not ^`
- Disjunction for longer strings
  - `pipe`

# Disjunction Contd..

- Negation in Disjunction
  - `^ right after the square bracket means a negation`
  - `[^A-Z]`
  - `[^Aa] means neither a capital A nor a lowercase a`
  - `[^e^] means not an e, and not ^`
- Disjunction for longer strings
  - `pipe`
  - `a|b|c = [abc]`

# Disjunction Contd..

- Negation in Disjunction
  - `^ right after the square bracket means a negation`
  - `[^A-Z]`
  - `[^Aa] means neither a capital A nor a lowercase a`
  - `[^e^] means not an e, and not ^`
- Disjunction for longer strings
  - `pipe`
  - `a|b|c = [abc]`
  - `apple|pie`

# Disjunction Contd..

- Negation in Disjunction
  - `^ right after the square bracket means a negation`
  - `[^A-Z]`
  - `[^Aa] means neither a capital A nor a lowercase a`
  - `[^e^] means not an e, and not ^`
- Disjunction for longer strings
  - `pipe`
  - `a|b|c = [abc]`
  - `apple|pie`
  - `[aA]pple|[aA]nd`

# Special characters

- ? - previous atom or group is optional: colou?r - color, colour

# Special characters

- ? - previous atom or group is optional: colou?r - color, colour
- . matches any character except a newline by default e.g. beg.n matches begun, begin, began

# Special characters

- ? - previous atom or group is optional: colou?r - color, colour
- . matches any character except a newline by default e.g. beg.n matches begun, begin, began
- Kleene Operators - named after Stephen Kleene

# Special characters

- ? - previous atom or group is optional: colou?r - color, colour
- . matches any character except a newline by default
  e.g. beg.n matches begun, begin, began
- Kleene Operators - named after Stephen Kleene
  - * matches 0 or more repetitions of the preceding atom or group
    e.g. oo*h matches oh, ooh, oooh, etc.
    (abc)* will match abc, abcabc, etc.

# Special characters

- ? - previous atom or group is optional: colou?r - color, colour
- . matches any character except a newline by default
  e.g. beg.n matches begun, begin, began
- Kleene Operators - named after Stephen Kleene
  - * matches 0 or more repetitions of the preceding atom or group
    e.g. oo*h matches oh, ooh, oooh, etc.
    (abc)* will match abc, abcabc, etc.
  - + matches 1 or more repetitions of the preceding atom or group
    e.g. o+h matches oh, ooh, oooh, etc.

# Repetition Ranges

- Specific ranges can also be specified

# Repetition Ranges

- Specific ranges can also be specified
- `{ }` to specify range for the immediately preceding regex

# Repetition Ranges

- Specific ranges can also be specified
- `{ } to specify range for the immediately preceding regex`
- `{n}  means exactly n occurrences`

# Repetition Ranges

- Specific ranges can also be specified
- `{ } to specify range for the immediately preceding regex`
- `{n}  means exactly n occurrences`
- `{n,} means at least n occurrences`

# Repetition Ranges

- Specific ranges can also be specified
- `{ } to specify range for the immediately preceding regex`
- `{n}  means exactly n occurrences`
- `{n,} means at least n occurrences`
- `{n,m} means at least n and no more than m occurrences`

# Repetition Ranges

- Specific ranges can also be specified
- `{ } to specify range for the immediately preceding regex`
- `{n}  means exactly n occurrences`
- `{n,} means at least n occurrences`
- `{n,m} means at least n and no more than m occurrences`
- Example:

  `.{0,} is equivalent to .*`

# More Regex

- Anchors

# More Regex

- Anchors
  - `^` matches the start of the string (or a line in multiline mode) e.g.

    `^[A-Z] matches an ASCII capital at the start.`

# More Regex

- Anchors
  - `^` matches the start of the string (or a line in multiline mode) e.g.

    `^[A-Z] matches an ASCII capital at the start.`
  - `$` matches the end, or before a final newline; multiline mode also matches line ends.

# More Regex

- Anchors
  - `^` matches the start of the string (or a line in multiline mode) e.g.

    `^[A-Z] matches an ASCII capital at the start.`
  - `$` matches the end, or before a final newline; multiline mode also matches line ends.
- `\. means a period`

# More Regex

- Anchors
  - `^` matches the start of the string (or a line in multiline mode) e.g.

    `^[A-Z] matches an ASCII capital at the start.`
  - `$` matches the end, or before a final newline; multiline mode also matches line ends.
- `\. means a period`
- Example: look for the word ‘the’

# More Regex

- Anchors
  - `^` matches the start of the string (or a line in multiline mode) e.g.

    `^[A-Z] matches an ASCII capital at the start.`
  - `$` matches the end, or before a final newline; multiline mode also matches line ends.
- `\. means a period`
- Example: look for the word ‘the’
  - missed capitalization: [tT]he

# More Regex

- Anchors
  - `^` matches the start of the string (or a line in multiline mode) e.g.

    `^[A-Z] matches an ASCII capital at the start.`
  - `$` matches the end, or before a final newline; multiline mode also matches line ends.
- `\. means a period`
- Example: look for the word ‘the’
  - missed capitalization: [tT]he
  - make pattern more precise:

    `r"\b[tT]he\b"`

# False Positive and Negatives

- False positives or Type 1 errors - matching things we shouldn't match

# False Positive and Negatives

- False positives or Type 1 errors - matching things we shouldn't match
- False negatives or Type 2 errors - not matching things we should match

# False Positive and Negatives

- False positives or Type 1 errors - matching things we shouldn't match
- False negatives or Type 2 errors - not matching things we should match
- Cost attached to false negative and positive

# False Positive and Negatives

- False positives or Type 1 errors - matching things we shouldn't match
- False negatives or Type 2 errors - not matching things we should match
- Cost attached to false negative and positive
- Provide some metrics by comparing against good data for a small sample

# Edit Distance

- pwned -> owned or pawned?

# Edit Distance

- pwned -> owned or pawned?
- standd -> strand, stand, stood, or sand?

# Edit Distance

- pwned -> owned or pawned?
- standd -> strand, stand, stood, or sand?
- How similar are two strings?

# Edit Distance

- pwned -> owned or pawned?
- standd -> strand, stand, stood, or sand?
- How similar are two strings?
- Applications

# Edit Distance

- pwned -> owned or pawned?
- standd -> strand, stand, stood, or sand?
- How similar are two strings?
- Applications
    - Spell Correction

# Edit Distance

- pwned -> owned or pawned?
- standd -> strand, stand, stood, or sand?
- How similar are two strings?
- Applications
  - Spell Correction
  - Also comes up in computational biology

# Edit Distance

- pwned -> owned or pawned?
- standd -> strand, stand, stood, or sand?
- How similar are two strings?
- Applications
  - Spell Correction
  - Also comes up in computational biology
  - Machine translation

# Edit Distance

- pwned -> owned or pawned?
- standd -> strand, stand, stood, or sand?
- How similar are two strings?
- Applications
  - Spell Correction
  - Also comes up in computational biology
  - Machine translation
  - Information extraction

# Edit Distance

- pwned -> owned or pawned?
- standd -> strand, stand, stood, or sand?
- How similar are two strings?
- Applications
  - Spell Correction
  - Also comes up in computational biology
  - Machine translation
  - Information extraction
  - Speech recognition

# Edit Distance

- Typically refers to minimum edit distance

# Edit Distance

- Typically refers to minimum edit distance
- Minimum number of editing operations to convert one string to another

# Edit Distance

- Typically refers to minimum edit distance
- Minimum number of editing operations to convert one string to another
  - Insertion
  - Deletion
  - Substitution
- e.g. two strings: intention, execution

# Edit Distance

- Typically refers to minimum edit distance
- Minimum number of editing operations to convert one string to another
  - Insertion
  - Deletion
  - Substitution
- e.g. two strings: intention, execution
  - align it with second letter

# Edit Distance

- Typically refers to minimum edit distance
- Minimum number of editing operations to convert one string to another
  - Insertion
  - Deletion
  - Substitution
- e.g. two strings: intention, execution
  - align it with second letter
  - d (delete), s (substitute), s, i(nsert), s

# Edit Distance

- Typically refers to minimum edit distance
- Minimum number of editing operations to convert one string to another
  - Insertion
  - Deletion
  - Substitution
- e.g. two strings: intention, execution
  - align it with second letter
  - d (delete), s (substitute), s, i(nsert), s
  - with unit costs, Levenshtein distance = 5

# Edit Distance

- Typically refers to minimum edit distance
- Minimum number of editing operations to convert one string to another
  - Insertion
  - Deletion
  - Substitution
- e.g. two strings: intention, execution
  - align it with second letter
  - d (delete), s (substitute), s, i(nsert), s
  - with unit costs, Levenshtein distance = 5
  - with substitution cost 2 instead of 1, weighted edit distance = 8

# Edit Distance

- Typically refers to minimum edit distance
- Minimum number of editing operations to convert one string to another
  - Insertion
  - Deletion
  - Substitution
- e.g. two strings: intention, execution
  - align it with second letter
  - d (delete), s (substitute), s, i(nsert), s
  - with unit costs, Levenshtein distance = 5
  - with substitution cost 2 instead of 1, weighted edit distance = 8
- You can implement this at word level so Microsoft Corp. is 1 away from Microsoft.

# Text Processing

# Text as Data

- Bag of words assumption
  Lose word order

# Text as Data

- Bag of words assumption
  Lose word order
- Remove stop words:
  If, and, but, who, what, the, they, their, a, or, ...
  Be careful: one person's stopword is another's key term.

# Text as Data

- Bag of words assumption
  Lose word order
- Remove stop words:
  If, and, but, who, what, the, they, their, a, or, ...
  Be careful: one person's stopword is another's key term.
- (Same) Word: Stemming and Lemmatization
  Porter stemming: taxing $\rightarrow$ tax, taxes $\rightarrow$ tax, taxation $\rightarrow$ taxat, taxable $\rightarrow$ taxabl.

# Text as Data

- Bag of words assumption
  Lose word order
- Remove stop words:
  If, and, but, who, what, the, they, their, a, or, ...
  Be careful: one person's stopword is another's key term.
- (Same) Word: Stemming and Lemmatization
  Porter stemming: taxing $\rightarrow$ tax, taxes $\rightarrow$ tax, taxation $\rightarrow$ taxat, taxable $\rightarrow$ taxabl.
- Remove rare words
  Choose thresholds on training data; validate their effect.

# Text as Data

- Bag of words assumption
  Lose word order
- Remove stop words:
  If, and, but, who, what, the, they, their, a, or, ...
  Be careful: one person's stopword is another's key term.
- (Same) Word: Stemming and Lemmatization
  Porter stemming: taxing $\rightarrow$ tax, taxes $\rightarrow$ tax, taxation $\rightarrow$ taxat, taxable $\rightarrow$ taxabl.
- Remove rare words
  Choose thresholds on training data; validate their effect.
- Convert to lowercase, drop numbers, punctuation, etc.

# How?

A small example with an explicit stop-word set.

```
from nltk.stem import PorterStemmer
from nltk.tokenize import wordpunct_tokenize

text = "Taxing taxes, taxation and taxable income."
stop_words = {"and", "the", "a"}
words = wordpunct_tokenize(text.lower())
words = [w for w in words
         if w.isalpha() and w not in stop_words]
stemmer = PorterStemmer()
stems = [stemmer.stem(w) for w in words]
print(stems)
```

# To Matrices

A document-term matrix has one row per document and one column per vocabulary term. Counts are one possible cell value.

```
from collections import Counter
from nltk import bigrams

text_bigrams = list(bigrams(words))
counts = Counter(stems)
print(text_bigrams)
print(counts)
```